

Reflexión sobre la solución implementada

La solución desarrollada extiende el resaltador léxico realizado previamente para permitir el procesamiento de múltiples archivos fuente de Python. El programa recibe una lista de rutas que pueden corresponder tanto a archivos individuales como a directorios, identifica automáticamente los archivos con extensión .py y genera un archivo HTML para cada uno de ellos con el resaltado de sintaxis correspondiente. De esta forma, se automatiza el análisis de varios archivos sin necesidad de ejecutarlo manualmente para cada caso.

Para resolver el problema se implementaron dos enfoques distintos. La primera solución corresponde a una versión secuencial, en la que los archivos son procesados uno tras otro. La segunda corresponde a una versión paralela basada en hilos de ejecución, donde varios archivos pueden analizarse simultáneamente. Debido a que el procesamiento de cada archivo es independiente de los demás, el problema resulta adecuado para aplicar técnicas de paralelización. Esta estrategia permite aprovechar mejor los recursos del sistema y reducir el tiempo total requerido para completar el análisis.

El algoritmo principal utilizado para el resaltado léxico se basa en un autómata finito determinista (DFA) representado mediante una tabla de transiciones. El programa recorre cada carácter del archivo, determina su categoría y realiza las transiciones correspondientes entre estados hasta identificar los diferentes tipos de tokens, como palabras reservadas, identificadores, operadores, números, comentarios y cadenas de texto. Este enfoque permite realizar el análisis de manera eficiente y organizada.

Durante el desarrollo se siguieron las convenciones de codificación de Python mediante el uso de nombres descriptivos para funciones y variables, una estructura clara del programa, indentación consistente y comentarios que facilitan la comprensión del código. Además, se incorporó manejo básico de excepciones para evitar fallos inesperados durante la lectura y procesamiento de los archivos.

Para evaluar el desempeño de ambas soluciones se realizaron mediciones de tiempo de ejecución. Los resultados obtenidos permitieron comparar directamente la versión secuencial con la versión paralela y calcular el speedup correspondiente. En general, la versión paralela presentó mejores tiempos de ejecución al distribuir el trabajo entre varios hilos, reduciendo el tiempo total necesario para procesar el conjunto de archivos. Sin embargo, la mejora observada depende de factores como la cantidad de archivos, su tamaño y el costo asociado a la creación y sincronización de los hilos.

Desde el punto de vista teórico, la complejidad temporal del algoritmo es $O(N)$, donde N representa la cantidad total de caracteres procesados. Esto se debe a que cada carácter es analizado una sola vez mediante las transiciones del autómata. Al procesar múltiples archivos, la complejidad continúa siendo lineal respecto al número total de caracteres analizados. Los resultados experimentales obtenidos son consistentes con esta complejidad, ya que el tiempo de ejecución aumenta de manera aproximadamente proporcional al tamaño total de la entrada.

Esto demuestra que el comportamiento observado en las pruebas coincide con el esperado según el análisis teórico.

Finalmente, es importante considerar las implicaciones éticas de este tipo de tecnología. Las herramientas de análisis automático de código pueden utilizarse para mejorar procesos educativos, aumentar la productividad de los desarrolladores y automatizar tareas repetitivas. Sin embargo, también podrían emplearse para inspeccionar código sin autorización o monitorear de forma excesiva el trabajo de otras personas. Por ello, es fundamental que estas tecnologías se utilicen de manera responsable, respetando la privacidad, la propiedad intelectual y los principios éticos asociados al manejo de información digital.